

AURALIS

A Smart Cane That Remembers

Research, Ideation and Design Plan · Artificial & Neural Networks Capstone

1. What we are building

A smart white cane for visually impaired people, with hardware and a mobile web app working together. It must tell the user what obstacle is in front through multilingual voice (phone speaker or Bluetooth earphones), and it must **remember locations and hazards**. It must contain real neural networks (course requirement) and be novel enough for a research paper. Timeline: 2-3 days.

One-line claim: AURALIS turns every hazard it detects into a persistent, time-decaying memory and uses that memory to warn early and choose safer routes. Existing smart canes only react to what is in front of them right now.

2. Research: what already exists

Obstacle detection with sensors, cameras and YOLO is already well studied, so it is our **baseline**, not our novelty.

Existing work	What it does	What it lacks
WeWALK Smart Cane 2 (commercial, CES 2025)	Chest-height ultrasonic alerts, app, turn-by-turn navigation, AI assistant	No learned hazard memory; expensive
Stanford Augmented Cane (2021, \$400)	LiDAR, GPS, IMU, SLAM, motorised wheel steers user; +20% walking speed	Heavy, complex; no long-term hazard memory
Low-cost YOLO smart cane (MDPI Information, 2025)	Dual RGB cameras + ultrasonic, YOLOv5x/YOLOv8n, surface transition detection	Reactive only
CV + depth smart cane (arXiv 2025)	Camera + depth sensor real-time detection	Reactive only
LiDAR + RGB-D cane (PMC)	2D LiDAR, RGB-D, GPS, IMU, SLAM, improved YOLOv5	Costly sensors; no risk memory
Ultrasonic cane prototype (Springer 2025) , S-Cane	Ultrasonic distance + audio / Bluetooth app alerts	No vision, no memory
AI smart cane survey (2025) , navigation survey	Reviews of the field; crowdsourced hazard reporting mentioned as a direction	-

The gap: almost every system is reactive. It warns about what is in front now and forgets it immediately. None learns the user's routes with hazards that strengthen on repeated sightings and fade over time, and then uses that to route around danger. That gap is our research contribution.

3. Chosen features (1 + 2 + 3)

#	Feature	How it works	Why it is novel
1	Hazard memory that fades	Each confirmed static hazard is saved with GPS, time, class, confidence. Seen again -> stronger. Not seen -> decays (half-life ~7 days). User can say 'hazard gone'.	Warns ~15-25 m before the sensors can see it, vs ~1-2 m for reactive canes
2	Self-learning safest route	Walks are recorded as a path graph (no map data needed). Saved places by voice ('library'). A* route cost = length x (1 + lambda x risk).	Prefers a slightly longer, safer path; works on campuses that maps don't cover

#	Feature	How it works	Why it is novel
3	Neural sensor fusion	MLP combines camera class/confidence/box size/position, AI-estimated distance, ultrasonic drop reading, approach speed and memory risk into one alert level.	Learned fusion compared against if-else rules in the paper
+	Multilingual voice	Phone text-to-speech: English, Hindi, Telugu, Tamil (extendable), via speaker or Bluetooth earphones.	Accessibility for Indian users

Risk of a remembered hazard: $\text{Risk} = \text{Severity} \times (1 - e^{-\text{count}/2}) \times 0.5^{\text{age} / 7 \text{ days}} \times \text{Confidence}$

Three neural networks (course requirement): (1) a CNN detector (YOLO) fine-tuned on our obstacle classes, including potholes and drains; (2) a monocular depth-estimation network (Depth Anything V2 Small or MiDaS) that estimates distance from the single camera image, so no front distance sensors are needed; (3) a fusion MLP that outputs the alert level.

4. What counts as an obstacle

Definition: an obstacle is anything inside the user's **walking corridor** that could cause contact, a trip or a fall. The corridor is about 1 m wide, from the ground up to head height (about 2 m), and up to 4 m ahead. Objects outside the corridor are tracked silently and not announced, so the cane does not talk nonstop.

#	Category	Examples	Detected by	Saved to memory?
1	Drop / pit	pothole, open drain / manhole, step down, stairs down, curb edge	Ultrasonic sensor at tip (sudden longer reading) + camera	Yes (highest severity)
2	Raised ground	step up, stairs up, speed breaker, stones / debris	Camera + depth network + ultrasonic	Yes
3	Body-level static	pole, tree, wall, parked bike / car, bench, dustbin, barrier	Camera (YOLO) + depth network	Yes
4	Head-level	low branches, signboards, shop shutters, truck back-end	Camera angled to see up to head height + depth network	Yes (a normal cane misses these)
5	Moving	person, bicycle, motorbike, car, auto-rickshaw, bus, dog, cow	Camera + depth change between frames (approach speed)	No (announced only)
6	Zone / context (optional)	construction area, wet floor, crowd	Camera	Yes, as route-level risk

Detector labels. Already in pre-trained (COCO) models: person, bicycle, motorbike, car, bus, truck, dog, cow, bench, chair. **We must train:** pothole, open drain, stairs, curb, pole, auto-rickshaw, using public pothole/stairs datasets plus 100-200 of our own photos.

Alert levels

Level	When	Vibration	Voice example
Silent	Outside corridor, or > 4 m	none	-
Caution	1.5-4 m, or a remembered hazard ahead	1 short pulse	"Pole ahead, 3 metres"
Warning	0.75-1.5 m, or something approaching fast	2 pulses	"Bike on your left, 1 metre"
Stop	< 0.75 m, or a drop / pit at any distance	strong repeated pulses	"Stop. Pit ahead"

The same alert is not repeated within a few seconds (cooldown), and a higher level always interrupts a lower one.

5. Hardware (minimal)

Design principle: the camera does the seeing, AI does the measuring, the phone does the rest. The phone already has GPS, vibration, a microphone and Bluetooth earphones, so the cane itself needs very little. One cheap ultrasonic sensor is added for the most dangerous case: a drop or pothole the camera can miss in shadow or at night.

Part	Qty	Purpose	Approx. cost
Raspberry Pi 5 (4/8 GB) + 64 GB microSD + cooler (a Pi 4 also works, slower)	1	Runs YOLO + depth network, serves the web app over Wi-Fi	Rs. 7,000-9,000
Raspberry Pi Camera Module 3 (or a USB webcam)	1	Forward vision on the cane handle, tilted to cover ground to head height	Rs. 2,500-3,000
HC-SR04 ultrasonic sensor	1	Points down near the cane tip: detects drops, pits, potholes; backup if the camera fails	Rs. 80-100

Part	Qty	Purpose	Approx. cost
1 kOhm + 2 kOhm resistors	1 each	Voltage divider: HC-SR04 ECHO is 5 V, Pi GPIO is 3.3 V	Rs. 5
USB-C power bank (5 V 3 A, 10,000-20,000 mAh)	1	Powers the Pi	Rs. 1,200-1,800
Stick (PVC pipe or white cane) + mounts, tape, jumper wires	-	Body	Rs. 200-300
Phone + Bluetooth earphones (user's own)	1	GPS, voice, vibration, app, memory, routing	-

Total: about Rs. 11,000-14,000, almost all of it the Pi and camera.

Not needed (and why)

Dropped part	Replaced by
GPS module	Phone GPS
Front / head-level distance sensors (ToF)	Depth-estimation neural network on the camera image
IMU	Not essential; phone sensors can be used if needed
Haptic driver + vibration motors	Phone vibration + voice in earphones
Buttons	Earphone / phone buttons and voice commands

Ultrasonic wiring (HC-SR04 to Raspberry Pi)

HC-SR04 pin	Connect to
VCC	Pi 5 V (pin 2)
GND	Pi GND (pin 6)
TRIG	Pi GPIO23 (pin 16)
ECHO	Through 1 kOhm to Pi GPIO24 (pin 18); 2 kOhm from GPIO24 to GND

Mounting: sensor about 30-40 cm above the tip, angled 35-45 degrees down. It learns the normal ground distance while walking; a sudden longer reading means a drop or pit and triggers an immediate 'Stop' alert.

Fallback if no Pi is available: the phone's own camera runs the AI in the browser, and an ESP32 (about Rs. 400) reads the same HC-SR04 and sends it to the phone over Bluetooth.

6. Software

Layer	Choice
App	Mobile web app (Chrome on Android), installable as a PWA; works offline after first load
Detection	YOLOv8n / YOLO11n fine-tuned on Google Colab; exported for the Pi (or TensorFlow.js for the phone fallback)
Distance	Depth Anything V2 Small / MiDaS small for front distance; HC-SR04 for drops
Fusion	Small MLP (8 inputs -> 16 -> 16 -> 4 alert levels), trained in Python, runs on phone
Memory	On the phone (IndexedDB / localStorage): hazards, places, path graph
Routing	Risk-aware A* on the self-built path graph
Voice	Phone text-to-speech + speech recognition for naming places

Data flow: sense (camera + ultrasonic + phone GPS) -> detect (YOLO) + estimate depth (depth network) -> fuse (MLP) -> alert (voice + vibration) -> remember (hazard map) -> route (A*) -> learn (repeat sightings /

decay).

7. 3-day build plan

Day	Hardware	Software / AI	Paper
1	Get parts. Set up Pi + camera; wire the ultrasonic sensor; mount on stick; test drop detection	Collect 100-200 obstacle photos; label in Roboflow / CVAT	Related work section
2	Integrate camera; field walks on 2 campus routes	Fine-tune YOLO on Colab; build web app: voice, memory, places, routing	Method section
3	Final packaging; demo video	Train + evaluate fusion MLP; run experiments	Results, discussion, abstract

8. Experiments for the paper

#	Experiment	Metric
E1	Depth network vs tape-measured distance at 0.5 / 1 / 2 / 3 m	Mean absolute error
E1b	Drop detection: camera only vs camera + ultrasonic (steps, pits, shadows)	Detection rate, false alarms
E2	YOLO on our obstacle classes	Precision, recall, mAP@0.5
E3	Fusion MLP vs if-else rules	Accuracy, urgent-class recall, confusion matrix
E4	Early warning: walk a route 5 times with a fixed obstacle	Warning distance: reactive vs memory
E5	Shortest vs safest route for $\lambda = 0, 1, 2, 4, 8$	Path length vs accumulated risk
E6	Alert latency (obstacle appears -> voice starts)	Milliseconds
E7	Multilingual voice clarity, 5 listeners	Mean rating 1-5

9. Honest limitations

- Distance from a single camera (depth network) is approximate; good for alert bands, not exact measurement.
- Phone GPS is about +/-5-10 m: good for 'hazard ahead' memory, not for exact sidewalk steering.
- Custom classes (pothole, drain) need training data; accuracy depends on how many photos we collect.
- Web Bluetooth works in Chrome on Android, not iPhone Safari (only matters for the ESP32 fallback).
- Field testing with blindfolded sighted volunteers, not blind users, within the 3-day scope.

10. Open decisions

- Is a Raspberry Pi 5 (or Pi 4) and camera available?
- Languages: English, Hindi, Telugu, Tamil, or others?
- Rework the existing draft code in the auralis- repo, or start fresh?